MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

The RISC-V ABI

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- The RISC-V ABI
- Register naming conventions

The RISC-V ABI

- RISC-V has 32 general purpose registers
  - x0 to x31
- Registers can be freely read and written
  - except for x0!

The RISC-V ABI

- What happens when different people write code for the same machine?
- How can we make sure we correctly access system hardware and services?

The RISC-V ABI

- Application Binary Interfaces (ABIs) define sets of rules for these purposes
  - More on this in COMP15212
- RISC-V has its own!

Calling Conventions

- The ABI defines rules for the use of each register
  - This prevents compatibility issues
  - Improves program readability

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

The ABI Register Naming

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

Summary

- The RISC-V ABI defines a set of rules and calling conventions
- Registers assume specific roles and names
- Supports interoperability and readability

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Methods

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- What happens during a Python method call
- A basic solution
- Using a Stack
- Some RISC-V instructions and registers

Functions/Methods in Python

Declare function/method in one place:

```
def age (day, month, year) :
  ...
  ...
```

Use it from several places (possibly with different parameters):

```
myage = age (myday, mymonth, myyear);
yourage = age (yourday, yourmonth, youryear);
...
...
if myage > yourage:
   print("You are younger than me");
```

At the end of the method, which instruction gets executed next?

Functions/Methods in Python

Method Call:
(1) process method arguments
(2) remember where to restart
(3) branch to method
(8) use result

Use it from several places (possibly with different parameters):

```
myage = age (myday, mymonth, myyear); #method call
yourage = age (yourday, yourmonth, youryear); #method call
...
...
...
if myage > yourage:
   print("You are younger than me");
```

```
def age (day, month, year) : #Inside method
  ...
  ...
```

Inside method:
(4) use method parameters
(5) perform body of method to calculate the result
(6) put the result somewhere accessible
(7) branch back to restart point

The restart point is immediately after the call (3), ie (8):

Method Parameters and Return Values

- The method needs input parameters
  - Information from the calling code to be used by the method
- The method needs to be able to return a value
  - Information from the method back to the calling code

- **Most obvious way**: use of registers (RISC-V ABI names!)
  - Inputs: a0-a7 registers
  - Return: a0, a1 registers

How to get back to the right place

- Methods are called from multiple places
- We need to be able to return to the correct place after execution
  - The return point needs to be saved

```
jal  ra, func      # jump to the function code
...                # function body
jalr zero, ra, 0   # return to caller
```

How to get back to the right place

- Methods are c
- We need to b t place after execution
  - The retur

jal func

```
jal  ra, func       # jump to the function code
...                 # function body
jalr zero, ra, 0    # return to caller
```

jr ra

Using Pseudoinstructions

- Methods are called from multiple places
- We need to be able to return to the correct place after execution
  - The return point needs to be saved

```
call func    # jump to the function code
...          # function body
ret          # return to caller
```

ABI Registers

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

Complete example

```
myage = age (myday, mymonth, myyear)
#method call

  . . .

def age (day, month, year) :
#inside method

  . . .
  . . .
```

```
        lw a0, myday          ; (1) set up 1st argument
        lw a1, mymonth        ; (1) set up 2nd argument
        lw a2, myyear         ; (1) set up 3rd argument
        jal age               ; (2) save return point…
                              ; (3) …and call the method
        sw a0, myage, zero    ; (8) use the result

age     ...   ; (4) use parameters a0,a1,a2
        ...   ; (5) to calculate the result
        ...   ; (6) then put the result in a0
        jr ra ; (7) go back to calling point
```

Self-study: RISC-V code for this

```
a = add(c, d) #method call
. . .

def add (x, y) : #method declaration
      return x+y
```

Limitation

- What if the methods needs to use more registers?
  - Save the registers before method call and restore before return
  - We need a temporary space for this!
  - Let's use a data space

```
Age  sw s0, temp, zero
     . . .
     ; method body can use s0
     . . .
     lw s0, temp; recover the original value of s0
     jr ra
temp DEFW 0
```

What if a method calls another method?

```
e.g.

def age (day, month, year) :
# calculates age in days
# needs to call “daysinyear” for each year
# because that isn’t always the same number


def daysinyear (year) :
# depends if year is a leap year
```

What if a method calls another method?

```
        lw a0, myday
        lw a1, mymonth
        lw a2, myyear
        jal age ; (2) save return point in ra…
                ; (3) …and call method ‘age’
        sw a0, myage, zero

age     ...
        jal daysinyear ; (2) save return point in ra…
                       ; (3) …and call method ‘daysinyear’
        ...
        jr ra          ; (7) go back to calling point

Daysinyear             ; doesn’t call any method: a “leaf” (why?)

        ...
        jr ra          ; (7) go back to calling point
```

What could possibly go wrong with that?

What if a method calls another method?

```
        lw a0, myday
        lw a1, mymonth
        lw a2, myyear
        jal age ; (2) save return point in ra…
                ; (3) …and call method ‘age’
        sw a0, myage, zero

age     ...
        jal daysinyear ; (2) save return point in ra…
                       ; (3) …and call method ‘daysinyear’
        ..
        jr ra          ; (7) go back to calling point

Daysinyear             ; doesn’t call any method: a “leaf” (why?)

        ...
        jr ra          ; (7) go back to calling point
```

sw ra, temp, zero

lw ra, temp

temp DEFW 0

More Limitations

- The number of arguments is limited by the number of registers (a0-a7)
  - What do we do if we need more arguments?

- Methods can call themselves in a recursive way
  - Where do we store all these return addresses? Too many variables!

- Need an alternative to save and restore registers
  - A place where space can be made temporarily
  - A place where we don't need to label every location
  - A place where we can determine where things are

Stack

- A stack is like a pile of books
  - You can add anything on top (“push”)
  - You can remove anything from the top (“pop”)
  - Only the top is accessible

- This data structure is also knowns as LIFO = Last-In, First-Out

Stack

- Typically goes from higher memory address to lower
  - Adding to it decreases the address
  - Removing from it increases the address

- `sp` register points to the top
  - Decrease `sp` to push, increase to pop

	address	value
	00000000	????????
	????????	????????
	????????	????????
	????????	????????
Push↑	00001FF4	????????
SP→	00001FF8	00004400
Pop ↓	00001FFC	8632ACEF
stack base	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	FFFFFFFF	????????

Examples

```
li t0, 42
li t1, 43
```

- What does this do?
  - PUSH t0
  - PUSH t1
  - POP t0
  - POP t1

```
sw t0, 4[sp] ; PUSH t0
sw t1, 0[sp] ; PUSH t1
lw t0, 0[sp] ; POP t0
lw t1, 4[sp] ; POP t1
```

t0 = 43
t1 = 42

Examples

```
li t0, 42
li t1, 43
```

- What does this do?
  - PUSH t0
  - PUSH t1
  - POP t1
  - POP t0

```
sw t0, 4[sp] ; PUSH t0
sw t1, 0[sp] ; PUSH t1
lw t1, 0[sp] ; POP t1
lw t0, 4[sp] ; POP t0
```

t0 = 42
t1 = 43

Using the stack for method calls

- We can use the stack for:
  - Store parameters to pass to a method
  - Store return values (e.g. results) of a method
  - Store temporary variables (intermediate results)
  - Store return addresses for methods calling other methods
  - Store anything short-lived
  - Store registers we want to maintain outside the execution of the function (s0-s11 registers)

- Stack space exists only when in use!
  - Variables are permanently allocated

ABI Registers

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6-7	t1-t2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10-x11	a0-a1	Function arguments/return values
x12-x17	a2-a7	Function arguments
x18-x27	s2-s11	Saved Registers
x28-x31	t3-t6	Temporaries

Summary

- What happens during a Python method call
- Implementation using registers
- Use of a stack
- Some RISC-V instructions and conventions

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Manipulating the Stack

Luca Peres
luca.peres-2@manchester.ac.uk

Overview and Learning Outcomes

- Use of the stack
  - Parameters
  - Variables
  - Save registers
- Static variables
- More instructions

Recap on the stack

- PUSH adds to it
- POP removes from it

- `sp` register controls the stack
  - Top address
  - Decrease to push, increase to pop

	address	value
	00000000	????????
	????????	????????
	????????	????????
	????????	????????
Push↑	????????	????????
sp→	00001FF8	00004400
Pop ↓	00001FFC	8632ACEF
stack base	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	FFFFFFFF	????????

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

 subi sp, sp, 4
 sw   t0, 0[sp]        ;PUSH t0
 subi sp, sp, 4
 sw t1, 0[sp]          ;PUSH t1
 lw t0, 0[sp]          ;POP t0
 addi sp, sp , 4
 lw t1, 0[sp]          ;POP t1
 addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	????????
	00001FF4	????????
sp→	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

```
t0 = 0x00000042
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043
```

```
subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	????????
sp→	00001FF4	????????
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **First move sp to push a value**

```
t0 = 0x00000042
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	????????
sp→	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Store to to the stack address pointed by sp**

```
t0 = 0x00000042
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
sp→	00001FF0	????????
	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Decrement sp again to store t1**

```
t0 = 0x00000042
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
sp→	00001FF0	00000043
	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Store t1 to the stack**

```
t0 = 0x00000042
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
sp→	00001FF0	00000043
	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **POP always loads first**

t0 = **0x00000043**
t1 = 0x00000043

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	00000043
sp→	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Increment the value of sp to update the top of the stack**

```
t0 = 0x00000043
t1 = 0x00000043
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	00000043
sp→	00001FF4	00000042
	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Extract the second value**

```
t0 = 0x00000043
t1 = 0x00000042
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

 subi sp, sp, 4
 sw   t0, 0[sp]        ;PUSH t0
 subi sp, sp, 4
 sw t1, 0[sp]          ;PUSH t1
 lw t0, 0[sp]          ;POP t0
 addi sp, sp , 4
 lw t1, 0[sp]          ;POP t1
 addi sp, sp, 4
```

	address	value
	????????	????????
	????????	????????
	????????	????????
	00001FF0	00000043
	00001FF4	00000042
sp→	00001FF8	00004400
	00001FFC	8632ACEF
	00002000	ACFE2375
	????????	????????
	????????	????????
	????????	????????
	????????	????????

- **Update the top of the stack**

```
t0 = 0x00000043
t1 = 0x00000042
```

Recap on the stack

```
li t0, 0x00000042
li t1, 0x00000043

subi sp, sp, 4
sw   t0, 0[sp]        ;PUSH t0
subi sp, sp, 4
sw t1, 0[sp]          ;PUSH t1
lw t0, 0[sp]          ;POP t0
addi sp, sp , 4
lw t1, 0[sp]          ;POP t1
addi sp, sp, 4
```

Note that this code results in register swap
Extraction order matters!

```
t0 = 0x00000043
t1 = 0x00000042
```

Method calls (recap)

- where to put method arguments/parameters
- where to put restart point
- where to put result
- how to use arguments/parameters safely

Method Call:

(1) process method arguments
(2) remember where to restart
(3) branch to method
(8) use result

Inside method:

(4) use method parameters
(5) perform body of method to calculate the result
(6) put the result somewhere accessible
(7) branch back to restart point

Use of the stack for method calls

We can use the stack as a place to:

1. put parameters to pass them to a method
2. put the return address (ra) if a method calls another method
3. protect any registers that the method uses
4. put results to pass them back from a method
5. put temporary variables (intermediate results)
6. …put anything short-lived.

We only use stack space while it is in use,
whereas allocated space (temp) was permanently allocated

Storing the return address in the stack

```
age subi sp, sp, 4
    sw ra, 0[sp]    ;push ra to preserve it
    . . .           ;(put daysinyear's argument into a0)
    subi sp, sp, 4
    sw a0, 0[sp]    ;push the argument
    jal daysinyear
    lw a0, 0[sp]    ; Why do this?
    addi sp, sp, 4
    . . .
    lw ra, 0[sp]    ; … so we get the ra here.
    addi sp, sp, 4
    jr ra
```

Passing Parameters

So far, we used registers to pass parameters

a0 = 1st parameter, a1 = 2nd, a2 = 3rd etc.

- What if we have a lot of parameters (e.g. more than “a” registers)?
    - Practicality: How to assign/use registers when one method might call others?
    - Efficiency: is this the **best** use of registers?

Alternative: push arguments onto stack during call

Example call

```
myage = age (myday, mymonth, myyear)
```

```
subi sp, sp, 12
lw a0, myday
sw a0, 8[sp]
lw a0, mymonth
sw a0, 4[sp]
lw a0, myyear
sw a0, 0[sp]
jal age
sw a0, myage    ; Note - assumes the method returns the
                ; result in a0, as per the ABI convention
```

address	value
SP	myyear
SP+4	mymonth
SP+8	myday

Accessing stacked parameters

```
def age (int day, int month, int year) :
...


age ; parameters: use 8[sp], 4[sp] and 0[sp]
    ; e.g. lw a0, 4[sp]
    ; load mymonth into a0
    ; sp doesn't change!
    ...
    ...
    addi sp, sp, 12 ; discard 3 parameters
    jr ra           ; return
```

address	value
SP	myyear
SP+4	mymonth
SP+8	myday
SP+12	??

Storing the return address to the stack

If age calls another method:

- ra has to be pushed before any other method is called to protect the return address.
- This should be done first:

```
age subi sp, sp, 4
    sw ra, 0[sp] ; sp decreases by 4, parameters are now at:
    ;12[sp](day),8[sp](month) or 4[sp](year)
    ...
    jal other_method  ; this overwrites ra
    ...
    lw ra, 0[sp]      ; return,- puts the caller
    addi sp, sp, 16   ; ret into ra then discards
                      ; the parameters
```

address	value
SP	ra
SP+4	myyear
SP+8	mymonth
SP+12	myday

Local variables

Most variables declared inside a method have the same lifetime as the method itself:

- when the method is called, its variables are created
- when the method returns, its variables are destroyed (there are exceptions, but we will ignore them for now)

The most space-efficient way to do this is:

- at the method start, “push” extra space for its variables
- at the method end, “pop” the extra space

```
subi sp, sp, bytes of variables ; create space
. . .
addi sp, sp, bytes of variables ; destroy space
```

Saving Registers

If a method is complicated it may need to use some registers to evaluate results.

Some code (the “caller”) will have called the method (the “callee”).

What if the caller had important data in those registers?

We should protect that data before using the registers.

Two approaches:

“Caller saved” the caller should protect any values in registers that matter before calling a function, in case the registers are overwritten in the callee.

“Callee saved” the callee should protect the values in any registers before it makes use of them, in case they are important for the caller.

Caller and Callee saved registers

The caller and the callee know nothing about the code inside each other.

Caller saved: the caller has to push all registers that contain anything that it is using in case the callee overwrites them.

- but the callee may not use any of those registers, so pushing might be wasted effort

Callee saved: the callee should push the values in any registers that it uses before it uses them, they may be important for the caller.

- but the caller may not actually be using those registers, so pushing them might be wasted effort

Preference: Callee saved – methods should be “well behaved”

Methods should only do what they say they do and should not have any other impact on any other code or systems.

Stack Frame

We now have multiple possible uses for our stack (some optional). An example of what goes where:

Address	value
sp	method variables … … …
	saved registers … …
	ra
	Parameters … …
	method variables … …
	saved registers … …
	ra

pushed/ created by method-2

discarded by method-2

pushed/ created by method-1

Main code calls method-1 and method-1 calls method-2

Let’s create this stack frame

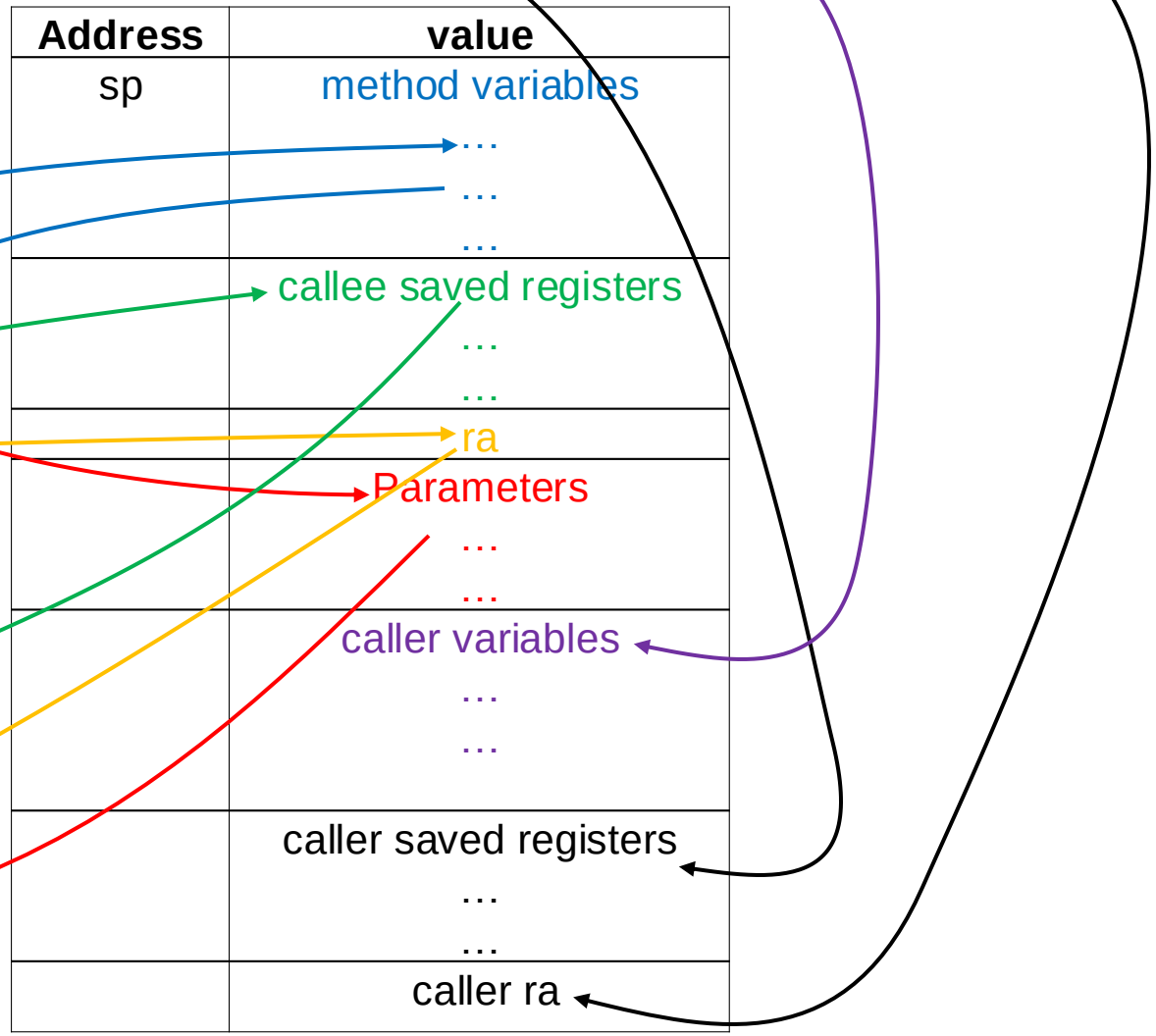
```
me   ... ;a method that finds out things about me
     subi sp, sp, 16
     sw ra, 12[sp]        ;push ra - me will call other methods
     sw a0, 8[sp]         ;push any registers used by ‘me’ (a0-a2)
     sw a1, 4[sp]
     sw a2, 0[sp]
     subi sp, sp, 36      ;create 9 variables for ‘me’
     ...
     subi sp, sp, 12
     sw a0, 8[sp]         ;push the three parameters for age
     sw a1, 4[sp]
     sw a2, 0[sp]
     jal age              ; method call
next ...

age                       ; a method that finds a person’s age
     subi, sp, sp, 24
     sw ra, 20[sp]        ; push ra - age will call other methods
     sw a0, 16[sp]        ; age uses a0-a4, protect for caller
     ...
     sw a4, 0[sp]
     subi sp, sp, 48      ; create 12 variables for ‘age’
     ...                  ; calculating age
     jal SomeOtherMethod  ; calculating age
     ...                  ; calculating age
     addi sp, sp, 48      ; discard the variable space
     lw a4, 0[sp]         ; recover the protected registers and ra
     ...
     lw a0, 16[sp]
     lw ra, 20[sp]
     addi sp, sp, 36      ; discard the space for the protected registers and for the parameters
     jr ra                ; return
```

Stack frame in 'age'

```
me    subi sp, sp, 16
      sw ra, 12[sp]
      sw a0, 8[sp]
      sw a1, 4[sp]
      sw a2, 0[sp]
      subi sp, sp, 36
      ...
      subi sp, sp, 12
      sw a0, 8[sp]
      sw a1, 4[sp]
      sw a2, 0[sp]
      jal age
next  ...

age
      subi, sp, sp, 24
      sw ra, 20[sp]
      sw a0, 16[sp]
      ...
      sw a4, 0[sp]
      subi sp, sp, 48
      ...
      jal SomeOtherMethod
      ...
      addi sp, sp, 48
      lw a4, 0[sp]
      ...
      lw a0, 16[sp]
      lw ra, 20[sp]
      addi sp, sp, 36
      jr ra
```



Example method call

```
def printAgeHistory (birthDay,
        birthMonth, birthYear) :
   someYear = birthYear + 1
   ageInSomeYear = 1
    . . .
```

Before the start of `printAgeHistory`:

sp	birthYear
sp+4	birthMonth
sp+8	birthDay

Example method call

```
def printAgeHistory (birthDay,
        birthMonth, birthYear) :
   someYear = birthYear + 1
   ageInSomeYear = 1
    . . .
```

sp	birthYear
sp+4	birthMonth
sp+8	birthDay

```
subi sp, sp, 4
sw ra, 0[sp] ; save ra
```

sp	ra
sp+4	birthYear
sp+8	birthMonth
sp+12	birthDay

Example method call

```
def printAgeHistory (birthDay,
        birthMonth, birthYear) :
   someYear = birthYear + 1
   ageInSomeYear = 1
    . . .
```

sp	ra
sp+4	birthYear
sp+8	birthMonth
sp+12	birthDay

```
subi sp, sp, 4
sw a0, 0[sp] ; save a0
```

sp	a0
sp+4	ra
sp+8	birthYear
sp+12	birthMonth
sp+16	birthDay

Example method call

```
def printAgeHistory (birthDay,
        birthMonth, birthYear) :
   someYear = birthYear + 1
   ageInSomeYear = 1
    . . .
```

sp	a0
sp+4	ra
sp+8	birthYear
sp+12	birthMonth
sp+16	birthDay

```
subi sp, sp, 8 ; create space for 2 variables
```

sp	ageInSomeYear
sp+4	someYear
sp+8	a0
sp+12	ra
sp+16	birthYear
sp+20	birthMonth
sp+24	birthDay

Example method call – method body

```
def printAgeHistory (birthDay,
        birthMonth, birthYear) :
   someYear = birthYear + 1
   ageInSomeYear = 1
    . . .
```

sp	ageInSomeYear
sp+4	someYear
sp+8	a0
sp+12	ra
sp+16	birthYear
sp+20	birthMonth
sp+24	birthDay

```
lw a0, 16[sp] ; a0 = birthYear
addi a0, a0, 1   ; a0 = birthYear + 1
sw a0, 4[sp]  ;someYear = birthYear + 1

li a0, 1       ; a0 = 1
sw a0, [sp]     ;ageInSomeYear = 1

. . .
```

Example – remove variables space

sp	ageInSomeYear
sp+4	someYear
sp+8	a0
sp+12	ra
sp+16	birthYear
sp+20	birthMonth
sp+24	birthDay

```
addi sp, sp, 8   ; remove space for 2 variables
```

sp	a0
sp+4	ra
sp+8	birthYear
sp+12	birthMonth
sp+16	birthDay

Example – restore registers

sp	a0
sp+4	ra
sp+8	birthYear
sp+12	birthMonth
sp+16	birthDay

```
lw a0, 4[sp]   ; restore saved register a0
addi sp, sp, 4
```

sp	ra
sp+4	birthYear
sp+8	birthMonth
sp+12	birthDay

Example – return

sp	ra
sp+4	birthYear
sp+8	birthMonth
sp+12	birthDay

```
lw ra, 0[sp]   ; restore ra
addi sp, sp, 16 : discard the parameters
```

Global/Static Variables

In Python, variables that are “static”

- must exist throughout the run-time of the program
- can never have more than 1 copy

Simplest implementation: DEFW

- Consequence – global variables are held in memory, access may be slower.

Summary

- Using Stack-Frames for:
  - Method Parameters (Arguments)
  - Method Variables
  - Saving Registers
- Static Variables